

Natural Language Processing

05. Linear Models

Juan José Alegría

April 8, 2025

Table of contents

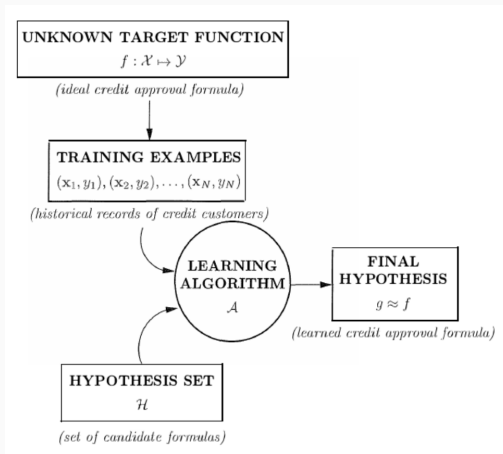
1. Supervised Learning
2. Linear models
3. Representations
4. Training
5. Regularization and hyperparameters
6. Limitations of linear models

Supervised Learning

Supervised Learning

- The essence of supervised machine learning is the creation of mechanisms that can look at examples and produce generalizations. [Goldberg, 2017]
- We design an algorithm whose input is a set of labeled examples, and its output is a function (or a program) that receives an instance and produces the desired label.
- Example: if the task is to distinguish from spam and not-spam email, the labeled examples are emails labeled as spam and emails labeled as not-spam.
- It is expected that the resulting function will produce correct label predictions also for instances it has not seen during training.
- This approach differs from designing an algorithm to perform the task (e.g., manually designed rule-based systems).

Supervised Learning

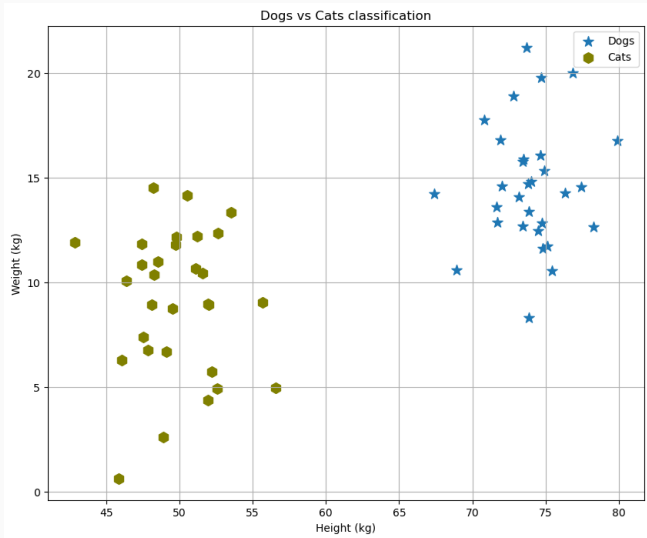


¹Source: [Abu-Mostafa et al., 2012]

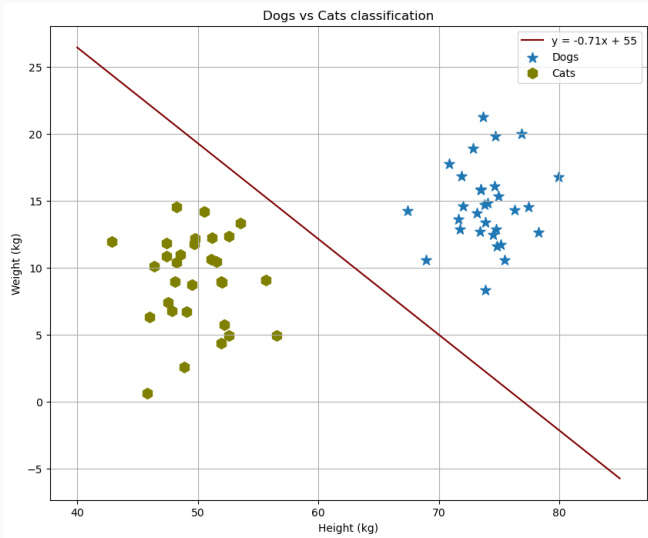
Parameterized Functions

- Searching over the set of all possible functions is a very hard (and rather ill-defined) problem. [Goldberg, 2017]
- We often restrict ourselves to search over specific families of functions.
- Example: the space of all linear functions with d_{in} inputs and d_{out} outputs,
- Such families of functions are called **hypothesis classes**.
- By restricting ourselves to a specific hypothesis class, we are injecting the learner with **inductive bias**.
- Inductive bias: a set of assumptions about the form of the desired solution.
- Some hypothesis classes facilitate efficient procedures for searching for the solution. [Goldberg, 2017]

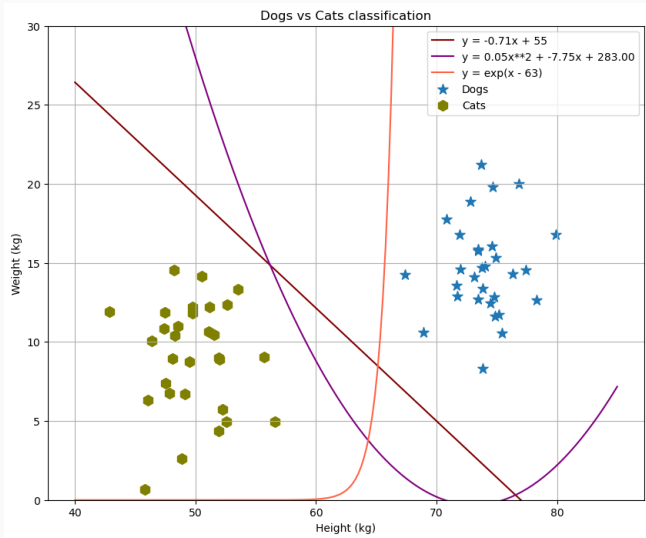
Classification problem



Classification problem: linear model



Classification problem: other alternatives



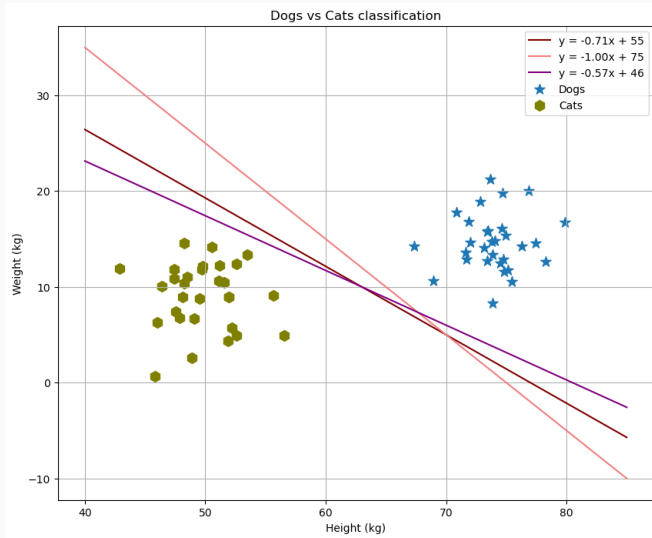
Linear models

- One common hypothesis class is that of high-dimensional linear function:

$$f(\vec{x}) = \vec{x} \cdot W + \vec{b}$$
$$\vec{x} \in \mathcal{R}^{d_{in}} \quad W \in \mathcal{R}^{d_{in} \times d_{out}} \quad \vec{b} \in \mathcal{R}^{d_{out}} \quad (1)$$

- The vector $\vec{x}$ is the input to the function.
- The matrix W and the vector $\vec{b}$ are the **parameters**.
- The goal of the learner is to set the values of the parameters W and $\vec{b}$ such that the function behaves as intended on a collection of input values $\vec{x}_{1:k} = \vec{x}_1, \dots, \vec{x}_k$ and the corresponding desired outputs $\vec{y}_{1:k} = \vec{y}_1, \dots, \vec{y}_k$
- The task of searching over the space of functions is thus reduced to one of searching over the space of parameters. [Goldberg, 2017]

Classification problem: linear model (several alternatives)



Example: Language Detection

- Consider the task of distinguishing documents written in English from documents written in German.
- This is a binary classification problem

$$f(\vec{x}) = \vec{x} \cdot \vec{w} + b \quad (2)$$

$d_{out} = 1$, $\vec{w}$ is a vector, and b is a scalar.

- The range of the linear function is $[-\infty, \infty]$.
- In order to use it for binary classification, it is common to pass the output of $f(x)$ through the *sign* function, mapping negative values to -1 (the negative class) and non-negative values to +1 (the positive class).

Example: Language Detection

- Letter frequencies make for quite good predictors (features) for this task.
- Even more informative are counts of letter bigrams, i.e., pairs of consecutive letters.
- One may think that words will also be good predictors i.e., using a bag of word representation of documents.
- Letters, or letter-bigrams are far more robust.
- We are likely to encounter a new document without any of the words we observed in the training set.
- While a document without any of the distinctive letter-bigrams is significantly less likely. [Goldberg, 2017]

Example: Language Detection

- We assume we have an alphabet of 28 letters (a–z, space, and a special symbol for all other characters including digits, punctuations, etc.)
- Documents are represented as 28×28 dimensional vectors $\vec{x} \in \mathcal{R}^{784}$.
- Each entry $\vec{x}_{[i]}$ represents a count of a particular letter combination in the document, normalized by the document's length.
- For example, denoting by $\vec{x}_{ab}$ the entry of $\vec{x}$ corresponding to the letter bigram ab :

$$\vec{x}_{ab} = \frac{\#ab}{|D|} \quad (3)$$

where $\#ab$ is the number of times the bigram ab appears in the document, and $|D|$ is the total number of bigrams in the document (the document's length - 1).

Example: Language Detection

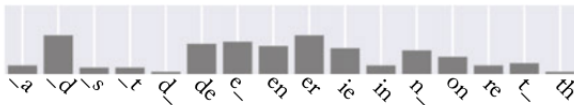


Character-bigram histograms for documents in English (left, blue) and German(right,green). Underscores denote spaces. Only the top frequent character-bigrams are shown.

²Source:[Goldberg, 2017]

Example: Language Detection

- Previous figure showed clear patterns in the data, and, given a new item, such as:



- We could probably tell that it is more similar to the German group than to the English one (observe the frequency of “th” and “ie”).
- We can’t use a single definite rule such as “if it has th its English” or “if it has ie its German”.
- While German texts have considerably less “th” than English, the “th” may and does occur in German texts, and similarly the “ie” combination does occur in English.

Example: Language Detection

- The decision requires weighting different factors relative to each other.
- We can formalize the problem in a machine-learning setup using a linear model:

$$\begin{aligned}\hat{y} &= \text{sign}(f(\vec{x})) = \text{sign}(\vec{x} \cdot \vec{w} + b) \\ &= \text{sign}(\vec{x}_{aa} \times \vec{w}_{aa} + \vec{x}_{ab} \times \vec{w}_{ab} + \vec{x}_{ac} \times \vec{w}_{ac} \cdots + b)\end{aligned}\tag{4}$$

- A document will be considered English if $f(\vec{x}) \geq 0$ and as German otherwise.

Intuition

1. Learning should assign large positive values to $\vec{w}$ entries associated with letter pairs that are much more common in English than in German (i.e., “th”).
2. It should also assign negative values to letter pairs that are much more common in German than in English (“ie”, “en”).
3. Finally, it should assign values around zero to letter pairs that are either common or rare in both languages.

Log-linear Binary classification

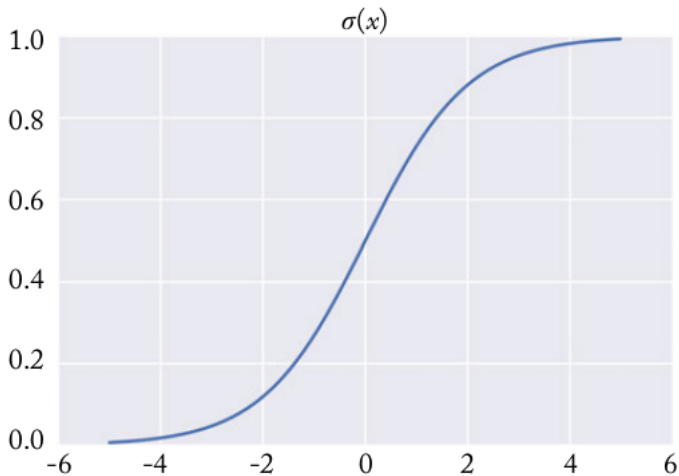
- The output $f(\vec{x})$ is in the range $[-\infty, \infty]$, and we map it to one of two classes $\{-1, +1\}$ using the *sign* function.
- This is a good fit if all we care about is the assigned class.
- We may be interested also in the confidence of the decision, or the probability that the classifier assigns to the class.
- An alternative that facilitates this is to map instead to the range $[0, 1]$, by pushing the output through a squashing function such as the sigmoid $\sigma(x)$:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (5)$$

resulting in:

$$\hat{y} = \sigma(f(\vec{x})) = \frac{1}{1 + e^{-\vec{x} \cdot \vec{w} - b}} \quad (6)$$

The Sigmoid function



The Sigmoid function

- The sigmoid function is monotonically increasing, and maps values to the range $[0, 1]$, with 0 being mapped to $\frac{1}{2}$.
- When used with a suitable loss function (discussed later) the binary predictions made through the log-linear model can be interpreted as class membership probability estimates:

$$\sigma(f(\vec{x})) = P(\hat{y} = 1|\vec{x}) \quad \text{of } \vec{x} \text{ belonging to the positive class.} \quad (7)$$

- We also get $P(\hat{y} = 0|\vec{x}) = 1 - P(\hat{y} = 1|\vec{x}) = 1 - \sigma(f(\vec{x}))$
- The closer the value is to 0 or 1 the more certain the model is in its class membership prediction, with the value of 0.5 indicating model uncertainty.
- Usually, we set a threshold $T = 0.5$. If $P(\hat{y}|\vec{x}) > T$, then we say that $\hat{y} = 1$, 0 otherwise. **Very important:** $T = 0.5$ is just the default threshold, but it can (and must!) be adjusted depending on the problem you are trying to solve; sometimes a false negative is more important than a false positive or vice versa.

Multi-class Classification

- Most classification problems are of a multi-class nature: examples are assigned to one of k different classes.
- Example: we are given a document and asked to classify it into one of six possible languages: English, French, German, Italian, Spanish, Other.
- Possible solution: consider six weight vectors $\vec{w}_{EN}, \vec{w}_{FR}, \dots$ and biases (one for each language).
- Predict the language resulting in the highest score:

$$\hat{y} = f(\vec{x}) = \operatorname{argmax}_{L \in \{EN, FR, GR, IT, SP, O\}} \vec{x} \cdot \vec{w}_L + b_L \quad (8)$$

Multi-class Classification

- The six sets of parameters $\vec{w}_L \in \mathcal{R}^{784}$ and b_L can be arranged as a matrix $W \in \mathcal{R}^{784 \times 6}$ and vector $\vec{b} \in \mathcal{R}^6$, and the equation re-written as:

$$\begin{aligned}\vec{\hat{y}} &= f(\vec{x}) = \vec{x} \cdot W + \vec{b} \\ \text{prediction} = \hat{y} &= \operatorname{argmax}_i \vec{\hat{y}}_{[i]}\end{aligned}\tag{9}$$

- Here $\vec{\hat{y}} \in \mathcal{R}^6$ is a vector of the scores assigned by the model to each language, and we again determine the predicted language by taking the argmax over the entries of $\vec{\hat{y}}$ (the columns with the highest value).

Representations

- Consider the vector $\vec{\hat{y}}$ resulting from applying a trained model to a document.
- The vector can be considered as a representation of the document.
- It captures the properties of the document that are important to us: the scores of the different languages.
- The representation $\vec{\hat{y}}$ contains strictly more information than the prediction $\operatorname{argmax}_i \hat{y}_{[i]}$.
- Example: $\vec{\hat{y}}$ can be used to distinguish documents in which the main language is German, but which also contain a sizeable amount of French words.
- Clustering documents based on $\vec{\hat{y}}$ could help to discover documents written in regional dialects, or by multilingual authors.

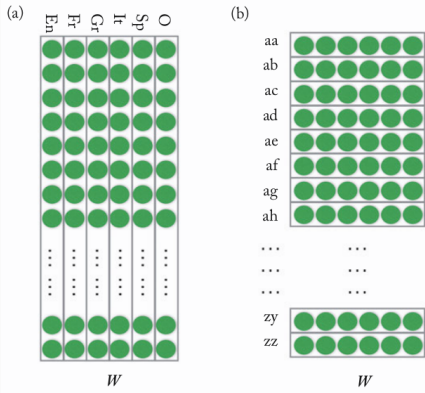
Representations

Representations

- The vectors $\vec{x}$ containing the normalized letter-bigram counts for the documents are also representations of the documents.
- Arguably containing a similar kind of information to the vectors $\vec{y}$.
- However, the representations in $\vec{y}$ is more compact (6 entries instead of 784) and more specialized for the language prediction objective.
- Clustering by the vectors $\vec{x}$ would likely reveal document similarities that are not due to a particular mix of languages, but perhaps due to the document's topic or writing styles.

Representations

- The trained matrix $W \in \mathcal{R}^{784 \times 6}$ can also be considered as containing learned representations.
- We can consider two views of W , as rows or as columns.



Two views of the W matrix. (a) Each column corresponds to a language. (b) Each row corresponds to a letter bigram. Source: [Goldberg, 2017].

- A column of W can be taken to be a 784-dimensional vector representation of a language in terms of its characteristic letter-bigram patterns.
- We can then cluster the 6 language vectors according to their similarity.
- Each of the 784 rows of W provide a 6-dimensional vector representation of that bigram in terms of the languages it prompts.

Representations

- Representations are central to deep learning.
- One could argue that the main power of deep-learning is the ability to learn good representations.
- In the linear case, the representations are interpretable.
- We can assign a meaningful interpretation to each dimension in the representation vector.
- For example: each dimension corresponds to a particular language or letter-bigram.

- Deep learning models, on the other hand, often learn a cascade of representations of the input that build on top of each other.
- These representations are often not interpretable.
- We do not know which properties of the input they capture.
- However, they are still very useful for making predictions.

One-Hot Vector Representation

- The input vector $\vec{x}$ in our language classification example contains the normalized bigram counts in the document D .
- This vector can be decomposed into an average of $|D|$ vectors, each corresponding to a particular document position i :

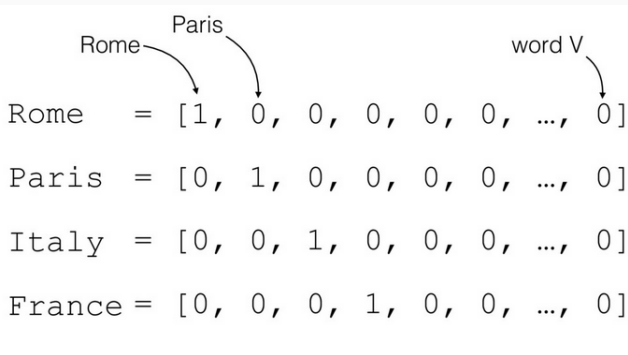
$$\vec{x} = \frac{1}{|D|} \sum_{i=1}^{|D|} \vec{x}^{D_{[i]}} \quad (10)$$

- Here, $D_{[i]}$ is the bigram at document position i .
- Each vector $\vec{x}^{D_{[i]}} \in \mathcal{R}^{784}$ is a one-hot vector.

One-Hot Vector Representation

- A one-hot vector: all entries are zero except the single entry corresponding to the letter bigram $D_{[i]}$, which is 1.
- The resulting vector $\vec{x}$ is commonly referred to as an averaged bag of bigrams (more generally averaged bag of words, or just bag of words).
- Bag-of-words (BOW) representations contain information about the identities of all the “words” (here, bigrams) of the document, without considering their order.
- A one-hot representation can be considered as a bag-of-a-single-word.

One-Hot Vector Representation



One-hot vectors of words. Source: <https://medium.com/@athif.shaffy/one-hot-encoding-of-text-b69124bef0a7>.

One-Hot Vector Representation

- From Eq. 9, we know that $\vec{\hat{y}} = f(\vec{x}) = \vec{x} \cdot W + \vec{b}$
- Let's forget for a minute about the biases $\vec{b}$ and focus only on $\vec{x} \cdot W$.
- Since $\vec{x}$ can be decomposed into a sum of one-hot vectors:

$$\begin{aligned}\vec{\hat{y}} &= \vec{x} \cdot W \\ &= \left(\frac{1}{|D|} \sum_{i=1}^{|D|} \vec{x}^{|D|_{[i]}} \right) \cdot W \\ &= \frac{1}{|D|} \left(\sum_{i=1}^{|D|} \vec{x}^{|D|_{[i]}} \cdot W \right) \\ &= \frac{1}{|D|} \sum_{i=1}^{|D|} W^{|D|_{[i]}}\end{aligned}\tag{11}$$

- In other words, the representation of the document can be understood as a sum of word representations. This is called *continous bag of words* (CBOW), and it will be revisited in a few classes from now :eyes:

Log-linear Multi-class Classification

- In the binary case, we transformed the linear prediction into a probability estimate by passing it through the sigmoid function, resulting in a log-linear model.
- The analog for the multi-class case is passing the score vector through the **softmax** function:

$$\text{softmax}(\vec{x})_{[l]} = \frac{e^{\vec{x}_{[l]}}}{\sum_j e^{\vec{x}_{[l]}}} \quad (12)$$

Resulting in:

$$\begin{aligned} \vec{\hat{y}} &= \text{softmax}(\vec{x} \cdot W + \vec{b}) \\ \vec{\hat{y}}_{[l]} &= \frac{e^{(\vec{x} \cdot W + \vec{b})_{[l]}}}{\sum_j e^{(\vec{x} \cdot W + \vec{b})_{[l]}}} \end{aligned} \quad (13)$$

- The softmax transformation forces the values in $\vec{\hat{y}}$ to be positive and sum to 1, making them interpretable as a probability distribution.

Training

- When training a parameterized function (e.g., a linear model, a neural network) one defines a loss function $L(\hat{y}, y)$, stating the loss of predicting $\hat{y}$ when the true output is y .

$$L(f(\vec{x}; \Theta), y)$$

- We use the symbol Θ to denote all the parameters of the model (e.g., $W, \vec{b}$)
- The training objective is then to minimize the loss across the different training examples.
- Formally, a loss function $L(\hat{y}, y)$ assigns a numerical score (a scalar) to a predicted output $\hat{y}$ given the true expected output y .

- The loss function should attain its minimum value for cases where the prediction is correct.
- We can also define a corpus-wide loss with respect to the parameters Θ as the average loss over all training examples:

$$\mathcal{L}(\Theta) = \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i)$$

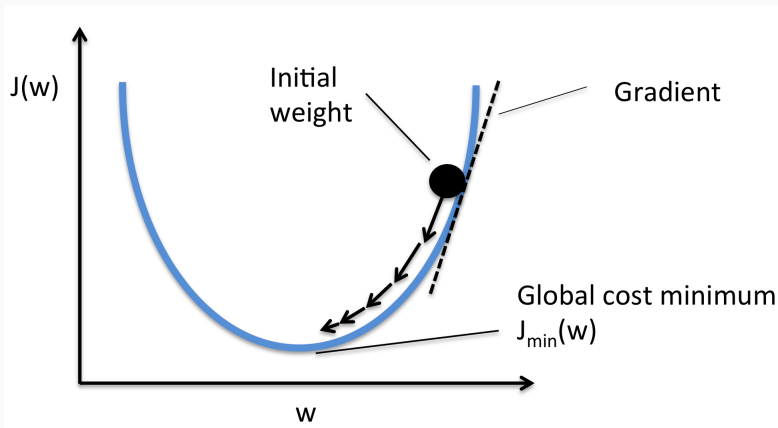
- The goal of the training algorithm is then to set the values of the parameters Θ such that the value of $\mathcal{L}$ is minimized.

$$\hat{\Theta} = \operatorname{argmin}_{\Theta} \mathcal{L}(\Theta) = \operatorname{argmin}_{\Theta} \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i)$$

Gradient-based Optimization

- Functions are trained using gradient-based methods.
- They work by repeatedly computing an estimate of the loss L over the training set.
- The training method computes gradients of the parameters (Θ) with respect to the loss estimate, and moving the parameters in the opposite directions of the gradient.
- Different optimization methods differ in how the error estimate is computed, and how moving in the opposite direction of the gradient is defined.
- If the function is convex, the optimum will be a global one.
- Otherwise, the process is only guaranteed to find a local optimum.

Gradient Descent



⁰Source: <https://sebastianraschka.com/images/faq/closed-form-vs-gd/ball.png>

Gradient Descent

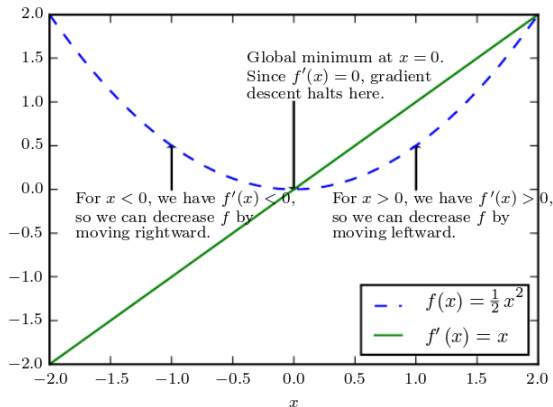


Figure 4.1: Gradient descent. An illustration of how the gradient descent algorithm uses the derivatives of a function to follow the function downhill to a minimum.

Online Stochastic Gradient Descent

- All the parameters are initialized with random values (Θ).
- For each training example (x, y) we calculate the loss L with current values of Θ .
- Then we update the parameters with the following rule until convergence:
- $\Theta_i \leftarrow \Theta_i - \eta \frac{\partial L}{\partial \Theta_i}(\hat{y}, y)$ (for all parameters Θ_i)

Algorithm 2.1 Online stochastic gradient descent training.

Input:

- Function $f(x; \Theta)$ parameterized with parameters Θ .
- Training set of inputs $x_1, \dots, x_n$ and desired outputs $y_1, \dots, y_n$.
- Loss function L .

```
1: while stopping criteria not met do
2:   Sample a training example  $x_i, y_i$ 
3:   Compute the loss  $L(f(x_i; \Theta), y_i)$ 
4:    $\hat{g} \leftarrow$  gradients of  $L(f(x_i; \Theta), y_i)$  w.r.t  $\Theta$ 
5:    $\Theta \leftarrow \Theta - \eta_i \hat{g}$ 
6: return  $\Theta$ 
```

⁰Source:[Goldberg, 2017]

Online Stochastic Gradient Descent

- The learning rate can either be fixed throughout the training process, or decay as a function of the time step t .
- The error calculated in line 3 is based on a single training example, and is thus just a rough estimate of the corpus-wide loss L that we are aiming to minimize.
- The noise in the loss computation may result in inaccurate gradients (single examples may provide noisy information).

Mini-batch Stochastic Gradient Descent

- A common way of reducing this noise is to estimate the error and the gradients based on a sample of m examples.
- This gives rise to the minibatch SGD algorithm

Algorithm 2.2 Minibatch stochastic gradient descent training.

Input:

- Function $f(x; \Theta)$ parameterized with parameters Θ .
- Training set of inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$ and desired outputs $y_1, \dots, y_n$.
- Loss function L .

```
1: while stopping criteria not met do
2:   Sample a minibatch of  $m$  examples  $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ 
3:    $\hat{\mathbf{g}} \leftarrow 0$ 
4:   for  $i = 1$  to  $m$  do
5:     Compute the loss  $L(f(\mathbf{x}_i; \Theta), y_i)$ 
6:      $\hat{\mathbf{g}} \leftarrow \hat{\mathbf{g}} + \text{gradients of } \frac{1}{m}L(f(\mathbf{x}_i; \Theta), y_i) \text{ w.r.t } \Theta$ 
7:    $\Theta \leftarrow \Theta - \eta_t \hat{\mathbf{g}}$ 
8: return  $\Theta$ 
```

- Higher values of m provide better estimates of the corpus-wide gradients, while smaller values allow more updates and in turn faster convergence.
- For modest sizes of m , some computing architectures (i.e., GPUs) allow an efficient parallel implementation of the computation in lines 3-6.

⁰Source:[Goldberg, 2017]

Loss Functions

- Hinge (or SVM loss): for binary classification problems, the classifier's output is a single scalar $\tilde{y}$ and the intended output y is in $\{+1, -1\}$. The classification rule is $\hat{y} = \text{sign}(\tilde{y})$, and a classification is considered correct if $y \cdot \tilde{y} > 0$.

$$L_{\text{hinge}(\text{binary})}(\tilde{y}, y) = \max(0, 1 - y \cdot \tilde{y})$$

- Binary cross entropy (or logistic loss): is used in binary classification with conditional probability outputs. The classifier's output $\tilde{y}$ is transformed using the sigmoid function to the range $[0, 1]$, and is interpreted as the conditional probability $P(y = 1|x)$.

$$L_{\text{logistic}}(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

Loss Functions

- The logistic loss has a probabilistic interpretation:
- We assume that $P(y = 1|\vec{x}; \Theta) = \sigma(f(\vec{x})) = \frac{1}{1+e^{-\vec{x} \cdot \vec{w} + b}}$ and $P(y = 0|\vec{x}; \Theta) = 1 - \sigma(f(\vec{x}))$
- We can write this in a more compact way:

$$P(y|\vec{x}; \Theta) = \sigma(f(\vec{x}))^y \times (1 - \sigma(f(\vec{x})))^{1-y}$$

- The above expression is the probability mass function of the Bernoulli distribution.
- Now we replace $\sigma(f(\vec{x}))$ by $\hat{y}$:

$$P(y|\vec{x}; \Theta) = \hat{y}^y \times (1 - \hat{y})^{1-y}$$

- If we perform maximum likelihood estimation to this expression we would apply a logarithm function and maximize the parameters Θ :

$$y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

- Maximizing this expression is equivalent to minimizing the logistic loss!
- Many loss functions correspond to the negative log-likelihood of probabilistic models!

Loss Functions

- Categorical cross-entropy loss: is used when a probabilistic interpretation of multi-class scores is desired. It measures the dissimilarity between the true label distribution $\vec{y}$ and the predicted label distribution $\vec{\hat{y}}$.

$$L_{\text{cross-entropy}}(\vec{\hat{y}}, \vec{y}) = - \sum_i \vec{y}_{[i]} \log(\vec{\hat{y}}_{[i]})$$

- When using the cross-entropy loss, it is assumed that the classifier's output is transformed using the softmax transformation.
- The softmax function squashes the k -dimensional output to values in the range $(0,1)$ with all entries adding up to 1. Hence, $\vec{\hat{y}}_{[i]} = P(y = i|x)$ represent the class membership conditional distribution.

- For hard-classification problems in which each training example has a single correct class assignment, $\vec{y}$ is a one-hot vector representing the true class.
- In such cases, the cross entropy can be simplified to:

$$L_{\text{cross-entropy(hard classification)}}(\vec{\hat{y}}, \vec{y}) = -\log(\hat{y}_{[t]})$$

where t is the correct class assignment.

Regularization and hyperparameters

Regularization

- Our optimization problem may admit multiple solutions, and, especially in higher dimensions, it can also over-fit.
- Scenario: In our language identification problem one of the documents in the training set ($\vec{x}_O$) is an outlier.
- The document is actually in German, but is labeled as French.
- In order to drive the loss down, the learner can identify features (letter bigrams) in $\vec{x}_O$ that occur in only few other documents.
- The learner will give these features very strong weights toward the (incorrect) French class.

- This is a bad solution to the learning problem.
- The model is learning something incorrect.
- Test German documents which share many words with $\vec{x}_O$ could be mistakenly classified as French.
- We would like to control for such cases by driving the learner away from such misguided solutions.
- Idea: it is OK to mis-classify a few examples if they don't fit well with the rest.

Regularization

- Regularization: we add a regularization term R to the optimization objective.
- The goal of this term: to control the complexity (large weights) of the parameter value (Θ), and avoid cases of overfitting:

$$\begin{aligned}\hat{\Theta} &= \operatorname{argmin}_{\Theta} \mathcal{L}(\Theta) + \lambda R(\Theta) \\ &= \operatorname{argmin}_{\Theta} \frac{1}{n} \sum_{i=1}^n L(f(\vec{x}_i; \Theta), y_i) + \lambda R(\Theta)\end{aligned}\tag{14}$$

- The regularization term considers the parameter values, and scores their complexity.
- The value of hyperparameter λ has to be set manually, based on the classification performance on a development set.

Regularization

- In practice, the regularizers R equate complexity with large weights.
- They work to keep the parameter values (Θ) low.
- They drive the learner toward solutions with low norms of the parameter matrices (W).
- Common choices for R are the L_2 norm, the L_1 norm, and the elastic-net.

L_2 Regularization

- In L_2 regularization, R takes the form of the squared L_2 norm of the parameters.
- Goal: try to keep the sum of the squares of the parameter values low.

$$R_{L_2}(W) = ||W||_2^2 = \sum_{i,j} (W_{[i,j]})^2$$

- The L_2 regularizer is also called a Gaussian prior or weight decay.
- L_2 regularized models are severely punished for high parameter weights.
- Once the value is close enough to zero, their effect becomes negligible.
- The model will prefer to decrease the value of one parameter with high weight by 1 than to decrease the value of ten parameters that already have relatively low weights by 0.1 each.

L_1 Regularization

- In L_1 regularization, R takes the form of the L_1 norm of the parameters.
- Goal: try to keep the sum of the absolute values of the parameters low.

$$R_{L_1}(W) = ||W||_1 = \sum_{i,j} |W_{[i,j]}|$$

- In contrast to L_2 , the L_1 regularizer is punished uniformly for low and high values.
- It has an incentive to decrease all the non-zero parameter values toward zero.
- It thus encourages a sparse solutions—models with many parameters with a zero value.
- The L_1 regularizer is also called a sparse prior or lasso [Tibshirani, 1996].

- The elastic-net regularization [Zou and Hastie, 2005] combines both L_1 and L_2 regularization:

$$R_{\text{elastic-net}}(W) = \lambda_1 R_{L_1}(W) + \lambda_2 R_{L_2}(W)$$

- While the SGD algorithm can and often does produce good results, more advanced algorithms are also available.
- The SGD+Momentum [Polyak, 1964] and Nesterov Momentum [Nesterov, 2018, Sutskever et al., 2013] algorithms are variants of SGD in which previous gradients are accumulated and affect the current update.
- Adaptive learning rate algorithms including AdaGrad [Duchi et al., 2011], AdaDelta [Zeiler, 2012], RMSProp [Tieleman and Hinton, 2012], and Adam [Kingma and Ba, 2014] are designed to select the learning rate for each minibatch.
- This sometimes done on a per-coordinate basis, potentially alleviating the need of fiddling with learning rate scheduling.
- For details of these algorithms, see the original papers or [Goodfellow et al., 2016](Sections 8.3, 8.4).

Train, Test, and Validation Sets

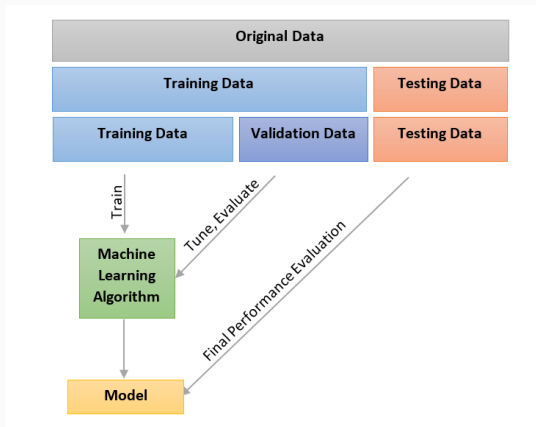
- When training a model our goal is to produce a function $f(\vec{x})$ that correctly maps inputs $\vec{x}$ to outputs $\hat{y}$ as evidenced by the training set.
- Performance on training data can be misleading: our goal is to train a function capable of generalizing to unseen examples.
- Held-out set: split training set into training and testing subsets (80% and 20% splits). Train on training and compute accuracy on testing.
- Problem: in practice you often train several models, compare their quality, and select the best one.
- Selecting the best model according to the held-out set's accuracy will result in an overly optimistic estimate of the model's quality.
- You don't know if the chosen settings of the final classifier are good in general, or are just good for the particular examples in the held-out sets.

Train, Test, and Validation Sets

- The accepted methodology is to use a three-way split of the data into train, validation (also called development), and test sets¹.
- This gives you two held-out sets: a validation set (also called development set), and a test set.
- All the experiments, tweaks, error analysis, and model selection should be performed based on the validation set.
- Then, a single run of the final model over the test set will give a good estimate of its expected quality on unseen examples.
- It is important to keep the test set as pristine as possible, running as few experiments as possible on it.
- Some even advocate that you should not even look at the examples in the test set, so as to not bias the way you design your model.

¹An alternative approach is cross-validation, but it doesn't scale well for training deep neural networks.

Train, Test, and Validation Sets



¹source:

<https://www.codeproject.com/KB/AI/1146582/validation.PNG>

Limitations of linear models

A limitation of linear models: the XOR problem

- The hypothesis class of linear (and log-linear) models is severely restricted.
- For example, it cannot represent the XOR function, defined as:

$$\begin{aligned}\text{xor}(0, 0) &= 0 \\ \text{xor}(1, 0) &= 1 \\ \text{xor}(0, 1) &= 1 \\ \text{xor}(1, 1) &= 0\end{aligned}\tag{15}$$

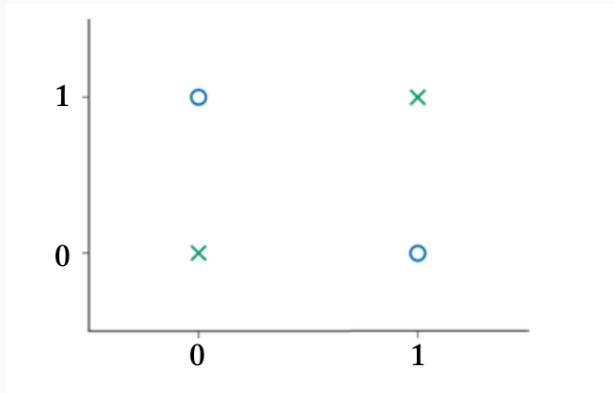
A limitation of linear models: the XOR problem

- There is no parameterization $\vec{w} \in \mathcal{R}^2, b \in \mathcal{R}$ such that:

$$\begin{aligned}(0, 0) \cdot \vec{w} + b &< 0 \\(0, 1) \cdot \vec{w} + b &\geq 0 \\(1, 0) \cdot \vec{w} + b &\geq 0 \\(1, 1) \cdot \vec{w} + b &< 0\end{aligned}\tag{16}$$

A limitation of linear models: the XOR problem

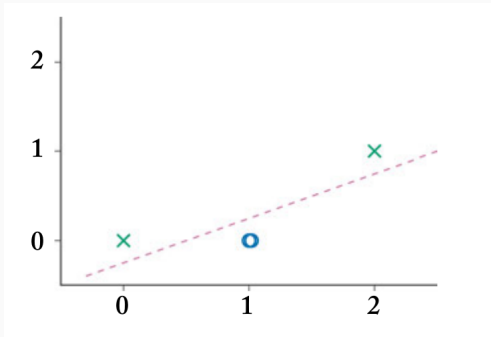
- To see why, consider the following plot of the XOR function, where blue Os denote the positive class and green Xs the negative class.



- It is clear that no straight line can separate the two classes.

Nonlinear input transformations

- If we transform the points by feeding each of them through the nonlinear function $\phi(x_1, x_2) = [x_1 \times x_2, x_1 + x_2]$, the XOR problem becomes linearly separable.



- The function ϕ mapped the data into a representation that is suitable for linear classification.

Nonlinear input transformations

- We can now easily train a linear classifier to solve the XOR problem.

$$\hat{y} = f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b \quad (17)$$

- Problem: we need to manually define the function ϕ .
- This process is dependent on the particular dataset, and requires a lot of human intuition.
- Solution: define a trainable nonlinear mapping function, and train it in conjunction with the linear classifier.
- Finding the suitable representation becomes the responsibility of the training algorithm.

Trainable mapping functions

- The mapping function can take the form of a parameterized linear model.
- Followed by a nonlinear activation function g that is applied to each of the output dimensions:

$$\begin{aligned}\hat{y} &= f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b \\ \phi(\vec{x}) &= g(\vec{x}W' + \vec{b}')\end{aligned}\tag{18}$$

Trainable mapping functions

- By taking $g(x) = \max(0, x)$ and $W' = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $\vec{b}' = \begin{pmatrix} -1 & 0 \end{pmatrix}$.
- We get an equivalent mapping to $[x_1 \times x_2, x_1 + x_2]$ for the our points of interest (0,0), (0,1), (1,0), and (1,1).
- This successfully solves the XOR problem!
- Learning both the representation function and the linear classifier on top of it at the same time is the main idea behind deep learning and neural networks.
- In fact, previous equation describes a very common neural network architecture called a multi-layer perceptron (MLP).

Thanks for your attention!



Abu-Mostafa, Y. S., Magdon-Ismail, M., and Lin, H.-T. (2012).

Learning From Data: A Short Course.

AMLBook.



Duchi, J., Hazan, E., and Singer, Y. (2011).

Adaptive subgradient methods for online learning and stochastic optimization.

Journal of Machine Learning Research, 12(Jul):2121–2159.



Goldberg, Y. (2017).

Neural network methods for natural language processing.

Synthesis Lectures on Human Language Technologies,
10(1):1–309.



Goodfellow, I., Bengio, Y., and Courville, A. (2016).

Deep learning.

MIT press.



Kingma, D. P. and Ba, J. (2014).

Adam: A method for stochastic optimization.

arXiv preprint arXiv:1412.6980.



Nesterov, Y. (2018).

***Lectures on convex optimization*, volume 137.**

Springer.



Polyak, B. T. (1964).

Some methods of speeding up the convergence of iteration methods.

USSR Computational Mathematics and Mathematical Physics, 4(5):1–17.



Sutskever, I., Martens, J., Dahl, G., and Hinton, G. (2013).
On the importance of initialization and momentum in deep learning.

In *International conference on machine learning*, pages 1139–1147.



Tibshirani, R. (1996).

Regression shrinkage and selection via the lasso.

Journal of the Royal Statistical Society: Series B (Methodological), 58(1):267–288.



Tieleman, T. and Hinton, G. (2012).

Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.

COURSERA: Neural networks for machine learning,
4(2):26–31.



Zeiler, M. D. (2012).

Adadelata: an adaptive learning rate method.

arXiv preprint arXiv:1212.5701.



Zou, H. and Hastie, T. (2005).

Regularization and variable selection via the elastic net.

Journal of the royal statistical society: series B (statistical methodology), 67(2):301–320.